

Section 4.3: Two Use Cases of Synthetic Data Generation for LLM

2026 Spring

[LLM Agents Foundation & Applications](#)

Dr. Yanjun Qi

20260220

What is Synthetic Data Generation (SDG)?

- A generative model **learns the statistical distribution** of real data, then samples brand-new artificial examples
- Generated data is **novel** — not a copy or a transformation of any real record
- It can be produced at **any scale**, for **any class**, under **any constraint**

The Core Problem: Real Data is Not Enough

- Privacy & Regulation — GDPR and sensitive PII restrict sharing medical, financial, or biometric data
- Scarcity & Cost — labeling real data is expensive; minority classes are underrepresented
- Bias & Imbalance — real-world datasets reflect historical biases that harm model fairness

Solution: Synthetic Data Generation (SDG) produces unlimited, pre-labeled, privacy-safe samples

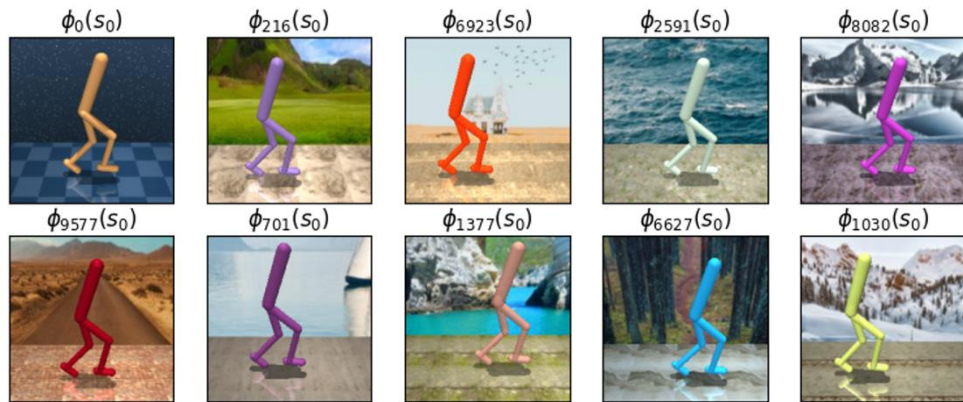
SDG is fundamentally different from data augmentation

- Data augmentation transforms existing, samples.
- SDG samples from a learned distribution to create genuinely new data.

SDG vs. Data Augmentation

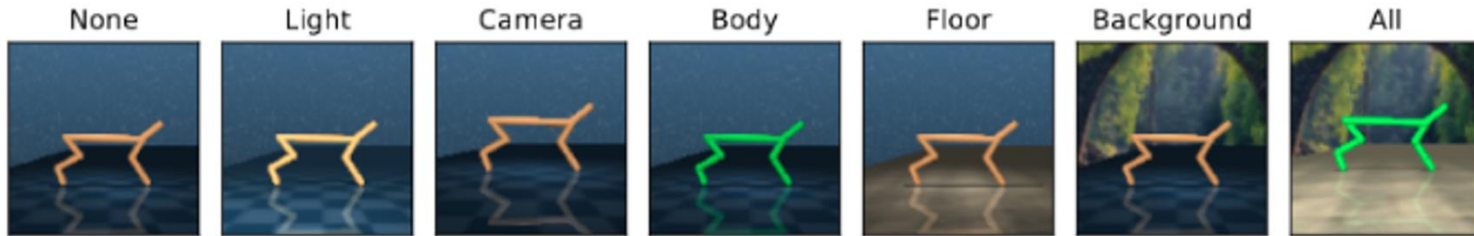
	Augmentation	SDG
Basis	Transforms existing samples	Samples a learned distribution
Output	Modified real data	Genuinely new data points
Privacy	Still uses real records	Can avoid real records entirely
Flexibility	Limited to transform types	Conditional on any attribute

Data Augmentation Examples

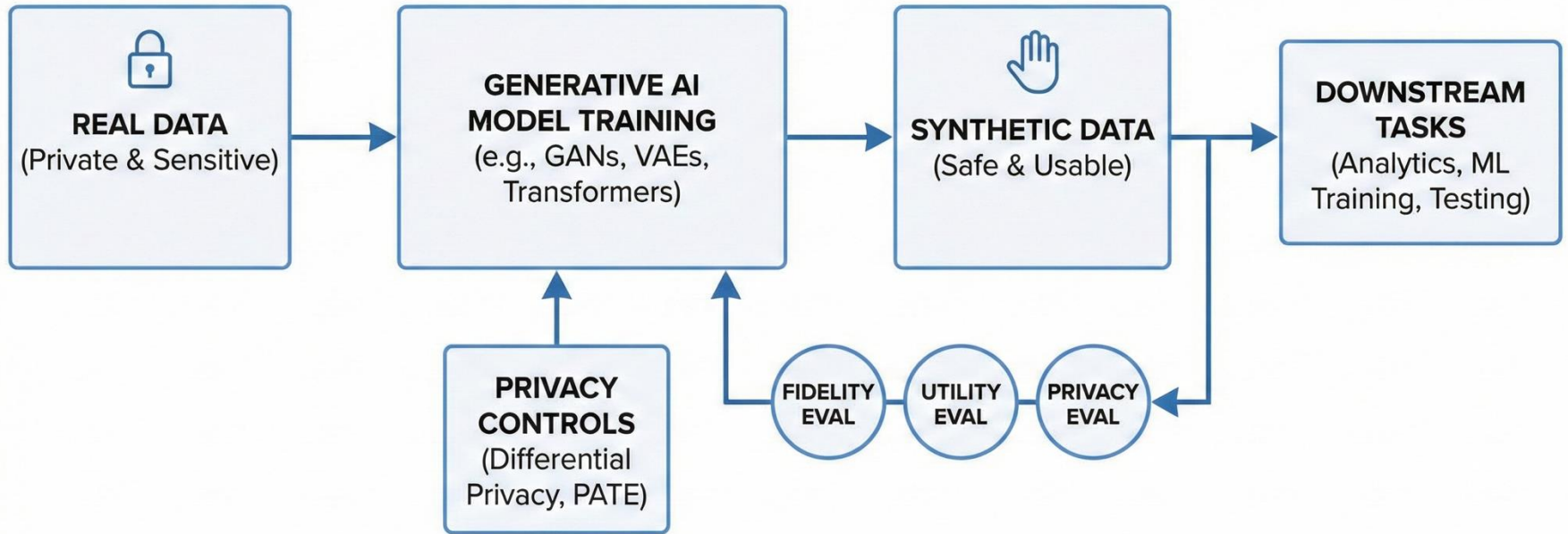


“Measuring Visual Generalization in Continuous Control from Pixels”

- With Background data augmentation



SYNTHETIC DATA GENERATION PIPELINE



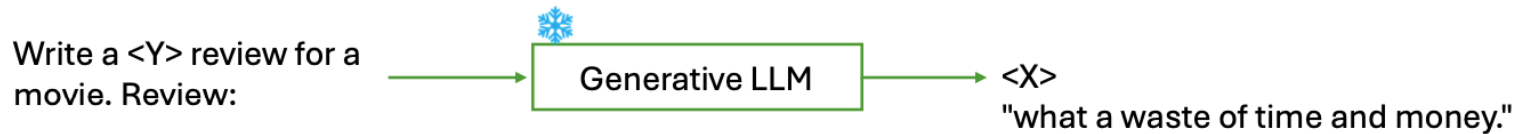
Data Modalities & Challenges

- **Tabular / Relational:** Structured records are difficult due to mixed data types, missingness, high-cardinality, and multi-table rules.
- **Time-Series:** Sequences with temporal correlation face challenges with irregular sampling and preserving event ordering.
- **Text:** Documents and clinical notes have a high risk of memorizing rare text spans like names or addresses.
- **Image / Video:** Perception stacks require perceptual realism and strict label fidelity.
- **Multimodal:** Sensor fusion or image+text requires cross-modal alignment.

LLM Based SDG

Guo & Chen (2024). "Generative AI for Synthetic Data Generation: Methods, Challenges and the Future."

LLM based SDG vs. Prompting



(Label-conditional prompts)

(a) Synthetic Data Generation



"what a waste of time and money."

(b) Prompting

But, Simple prompts are not enough

The naïve approach — *"Write a [label] review"* — has three known failure modes:

- **Low diversity** — the LLM reuses similar phrasing and sentence structures, producing near-duplicate samples
- **Label drift** — generated text does not always match the intended class, especially for subtle distinctions
- **Noisy training signal** — hallucinations and pre-training biases from the LLM propagate into the dataset

AIDE

AIDE: Attribute-Guided Multi-Hop Data Expansion for Data Scarcity in Task-Specific Fine-tuning

Fine-tune LLMs for any task — from just 10 seed examples.

Jiayu Li · Xuan Zhu · Fang Liu · Yanjun Qi

Syracuse University · AWS Bedrock Science

ACL 2025 ·

SDG Prompting with Rich Prompts:

Common Structure of LLM Prompts:

- [System/Role]
- [Task Instruction]
- [Context/Background]
- [Input Data]
- [Few Shot Examples]
- [Output Format]
- [Constraints]

**AIDE: Atttribute-Guided MultI-Hop Data Expansion
for Data Scarcity in Task-Specific Fine-tuning**

Jiayu Li^{1,3,*}, Xuan Zhu², Fang Liu², Yanjun Qi^{2,3}

¹Syracuse University, ²AWS Bedrock Science

³Correspondence: jli221@data.syr.edu, yanjunqi@amazon.com

Motivation: Task-Specific Fine Tuning needs data

- Most domains lack sufficient training data for **task-specific** fine-tuning LLMs
- Manual annotation is labor-intensive and costly across specialized fields
- Sampling from public NLP datasets covers limited, often irrelevant information
- Existing synthesis methods require large seed datasets (hundreds to thousands of examples)
- Generated data tends to lose task relevance OR diversity — rarely both are preserved

**Labeled task-specific data is scarce, expensive,
and hard to collect at scale.**

Challenges

Challenge:

How do we effectively fine-tune LLMs with few data to achieve good performance on a specific task?

Strategies:

Data augmentation/synthesis with few data.

Manually augmenting data is both time-consuming and labor-intensive.

➔ Solved by using LLMs to synthesize data.

Problem:

The existing methods either require a large set of seed data, or cannot simultaneously guarantee the relevance, diversity and high-quality of generated data.

Previous Methods

No existing method balances task relevance, diversity, and low seed requirements simultaneously.

Method	Seeds Needed	Task Relevance	Diversity
Evol-Instruct	Large (250K)	✗ Generic	✓
DataTune	Many datasets	Partial	Partial
Prompt2Model	Many datasets	✓	✗ Limited
AIDE (Ours)	10 seeds	✓	✓

Attribute-Guided Multi-Hop Data Expansion: AIDE

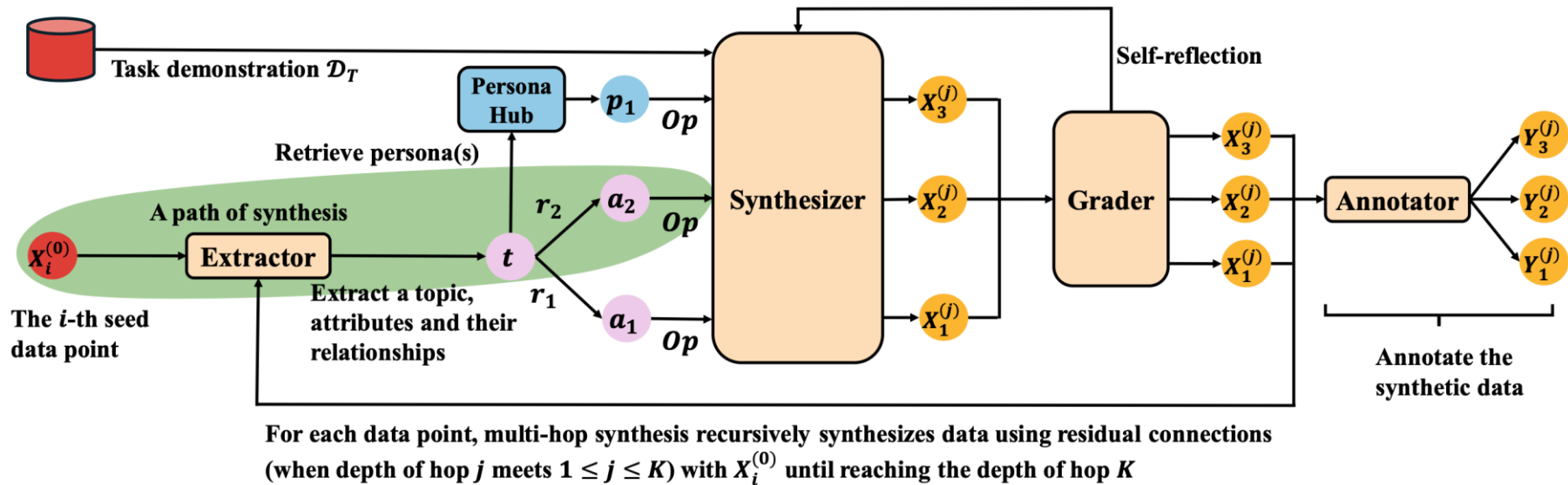


Figure 1: Overview of the workflow of AIDE. $x_i^{(0)}$ denotes the i -th task-related seed data point. AIDE includes four steps. (1) a LLM extractor extracts a topic t , knowledge attributes a_1 and a_2 with relationships r_1 and r_2 of a data point. (2) During the multi-hop synthesis at the depth of hop j , a LLM acts as a synthesizer with task demonstrations $\mathcal{D}_T$ to generate data $x_1^{(j)}$, $x_2^{(j)}$ and $x_3^{(j)}$ along paths of synthesis with a predefined operation Op (i.e., adding constraints). (3) To enhance the diversity of synthesis, we expand attributes by retrieving a persona p_1 from a persona hub with t . Finally, a LLM as an annotator generates the label of synthetic data. We describe the technical details of AIDE in Section 3.

Method: Components in AIDE

AIDE combines multi-hop synthesis, attribute/persona guidance, residual connections, and self-reflection.

- **Multi-Hop Synthesis:** Recursively expand each seed — each generated point becomes a new seed for the next hop
- **Attribute Guidance:** Extract knowledge triplets ⟨topic, relation, attribute⟩ to keep synthesis on-task
- **Persona Guidance:** Retrieve diverse personas from Persona Hub to diversify synthesis paths
- **Residual Connections:** Anchor deep hops back to the original seed to prevent topic drift at $K \geq 5$
- **Self-Reflection:** LLM grader scores relevance and diversity; low-scoring outputs are re-synthesized

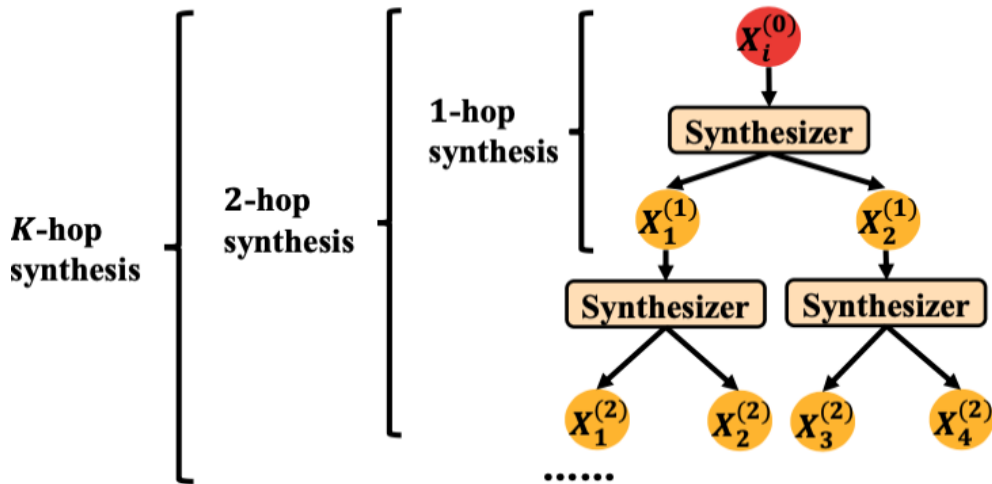
Multi-hop Synthesizer

With n limited seed data, we propose the K -hop synthesizer to synthesize abundant data.

Definition:

Given a seed data point $X_i^{(0)}$ where $0 < i \leq n$, K -hop synthesis involves using $(K - 1)$ -hop neighbors of $X_i^{(0)}$ to recursively generate m_K K -hop neighbors $X^{(K)} = \{X_1^{(K)}, X_2^{(K)}, \dots, X_{m_K}^{(K)}\}$ along certain paths. The i -th synthetic data can be described as $X_i^{(K)}$ where $0 < i \leq m_K$ and $m = n(1 + m_1 + m_2 + \dots + m_K)$.

Each seed branches
recursively — producing
a tree of ~3,000 training
examples from just 10
inputs.



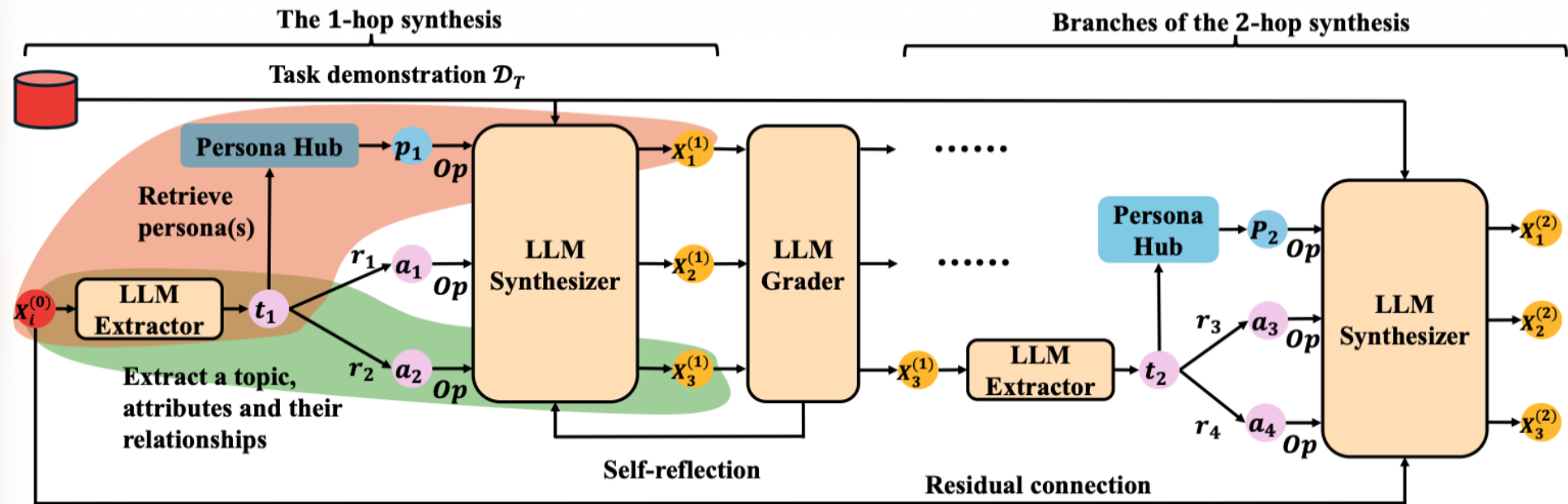


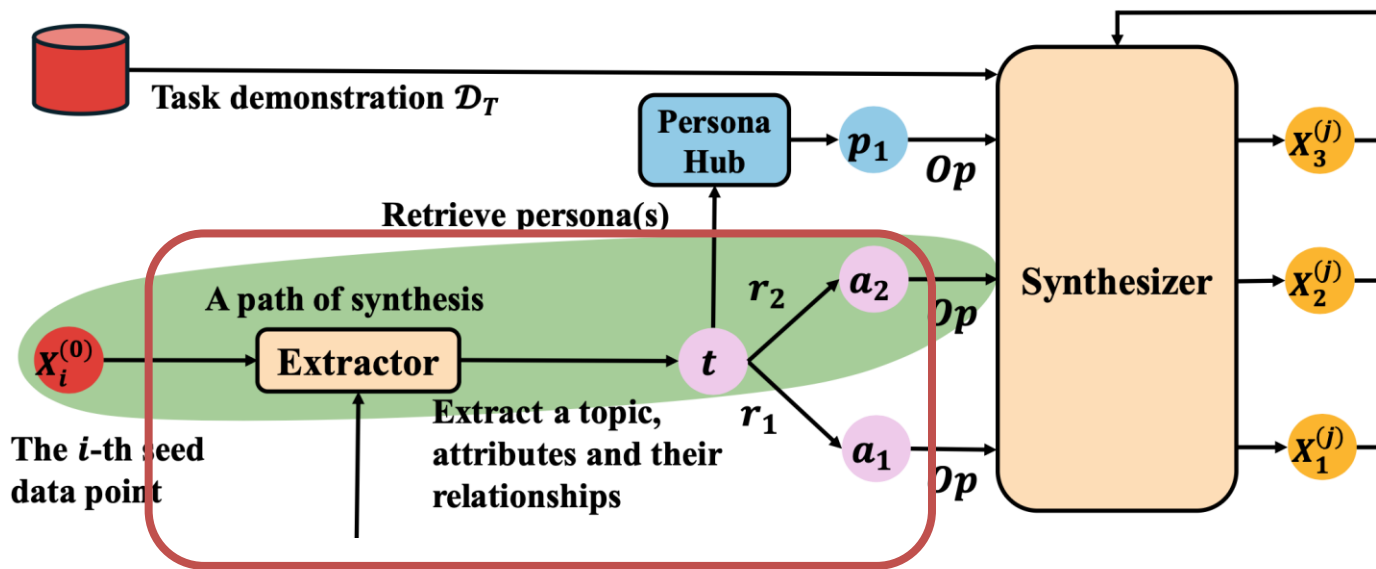
Figure 9: An example of unfolded multi-hop synthesis when $K = 2$.

Main Aim:

*How to **construct the paths** of **K-hop synthesis** such that we can **control the diversity** of synthetic data but still **relevant** to seed data during the data synthesis?*

K -hop Synthesis with Extracted Attributes

- **Relevant Topics via Extracting Attribute Knowledge Graph (KG) from Text**
 - A topic and related knowledge can be extracted from a text data point.
 - A knowledge graph (KG) consists of a topic and the related knowledge as entities.
 - Thus, a KG effectively serves as a representation of the text.



Example: Extract Attributes from Data

Prompt for extracting a topic and related knowledge of a data point

I want you to act as an instruction analyzer to generate A knowledge graph.

A knowledge graph can be represented by a triple [entity1, relation, entity2], which means entity1 has a relationship with entity2. Given an instruction, you should recognize its topic and related knowledge, which can be used to build a knowledge graph. The topic as entity1 and related knowledge as entity2 should have a specific relation. You need to list 3 related knowledge while each knowledge should be concise with one word or two words. Follow the examples below to analyze #The Given Instruction#.

#Example 1#

#The Given Instruction#: As a sports commentator, describe the winning play in the final seconds of a championship game.

#Thinking#: Topic is "creative writing" with related knowledge "role-play" and "sports". They have the relations ["creating writing", "uses", "role-play"] and ["creating writing", "involves", "sports"]

#Topic#: creative writing

#Related knowledge#: role-play, sports

... N Examples ...

#The Given Instruction#:

{Here is instruction.}

#Topic#:

#Related knowledge#:

#The Given Instruction#: Generate a list of ten items a person might need for a camping trip.

#Thinking#: Topic is "outdoor activities" with related knowledge "camping" and "equipment". They have the relations ["outdoor activities", "involves", "camping"] and ["outdoor activities", "requires", "equipment"]

#Topic#: outdoor activities
#Related knowledge#: camping, equipment

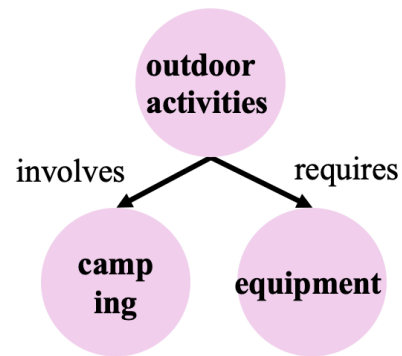


Figure 3. An example about extracting attributes from Data

K-hop Synthesis with Attribute (Related Knowledge)

- We extract $\mathcal{G}_i^{(K-1)}$ of a data point $X_i^{(K-1)}$ with a LLM
- We synthesize the data $X_i^{(K)}$ based on $X_i^{(K-1)}$ and a triplet $\langle t, r, k \rangle_i^{(K-1)}$ from a path of $\mathcal{G}_i^{(K-1)}$,

$$X_i^{(K)} = LLM(X_i^{(K-1)}, \langle t, r, k \rangle_i^{(K-1)}) \quad (2)$$

- A triplet ensures the topic and content of synthetic data remains relevant to those of input data,
- For further increasing complexity of synthetic data^[2], we introduce **predefined operations (Op)** following a triplet,

$$X_i^{(K)} = LLM(X_i^{(K-1)}, \langle t, r, k \rangle_i^{(K-1)}, Op) \quad (3)$$

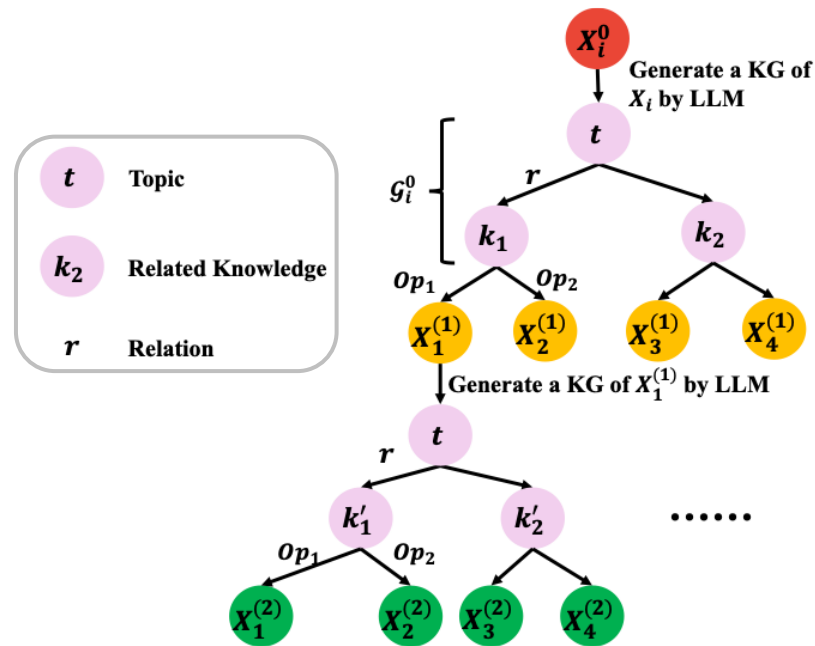


Figure 4: K-hop synthesis with Attribute KGs and predefined operations

Example: K -hop Synthesis with Attribute KG

Prompt for synthesis with a triplet of a KG and a constraint operation

I want you act as a Prompt Writer.

Your objective is to rewrite a given prompt into a more complex instruction to make those famous AI systems (e.g., chatgpt and GPT4) a bit harder to handle. But the rewritten prompt must be reasonable and must be understood and responded by humans. Please generate the #Rewritten Prompt# meets the following EXPECTATIONS:

EXPECTATION 1: The #Rewritten Prompt# can be obtained by adding simple constraints into content in #The Given Prompt#

EXPECTATION 2: The #Rewritten Prompt# is related to {Topic} and {Knowledge}.

EXPECTATION 3: You should not make the #Rewritten Prompt# become verbose. Follow the below examples to generate #Rewritten Prompt# by adding constraints into #The Given Prompt#.

{... N Examples ...}

#The Given Instruction#:

{Here is instruction.}

#Rewritten Prompt#:

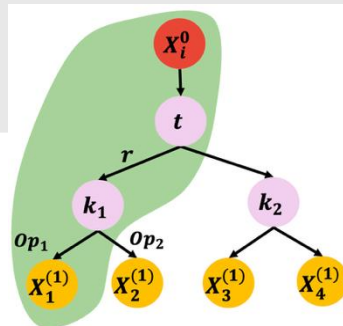


Figure 5. An example about K -hop Synthesis with KG.

#The Given Instruction#: Generate a list of ten items a person might need for a camping trip.

#Topic#: outdoor activities
#Related knowledge#: camping

1-hop synthesis using a triplet ["outdoor activities", "involves", "camping"] with an operation "adding constraint".

#Rewritten Prompt#: Generate a list of ten essential items required for a multi-day camping expedition, ensuring that the list includes both shelter and food.

K-hop Synthesis with Personas

Greater the diversity within a dataset is, the higher upper bound of performance of LLMs fine-tuned with the dataset can be achieved^[6].

Since a language model typically generates only one instance per synthesis prompt, achieving large-scale diversity demands a wide range of varied prompts^[7]

With **Persona Hub**^[7], for each data point, we employ personas to the construct paths for diversity of synthetic data.

We use a topic of a data point to retrieve top-K relevant personas in Persona Hub, denoted as $p_i \in P$ where p_i is the i -th persona in retrieved personas P .

With the personas and a data point $X_i^{(K-1)}$, we can synthesize $X_i^{(K)}$ as

$$X_i^{(K)} = LLM(X_i^{(K-1)}, t, p_i, Op) \quad (4)$$

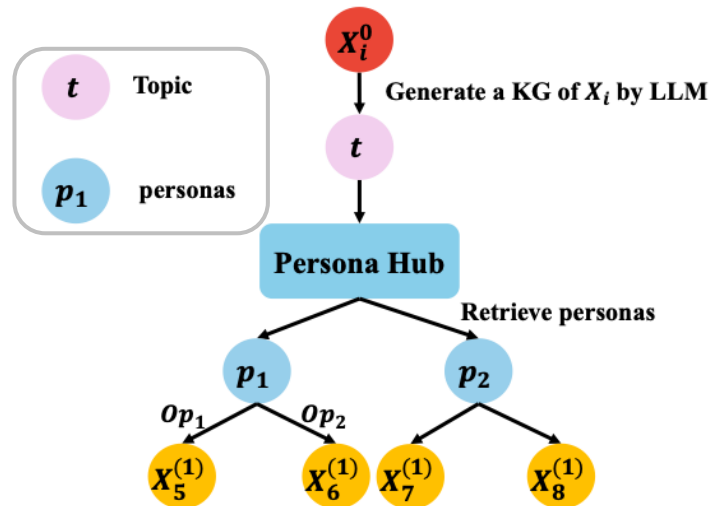


Figure 6: K-hop synthesis with personas and predefined operations

Example: K -hop Synthesis with Personas

Prompt for synthesis with a related persona

A persona is the aspect of someone's character.

You can use the given character to generate a #Created Prompt#. Your goal is to use #The Given Persona# to create a #Created Prompt# different from #The Given Prompt#. You SHOULD generate the #Created Prompt# through the following actions:

Action 1: #Created Prompt# can be any format but should be understood by humans.

Action 2: use the character from #The Given Persona# to generate #Created Prompt# by adding simple constraints in #The Given Prompt#.

Action 3: You should not make the #Created Prompt# become verbose. Follow the examples below to generate #Created Prompt#.

{... N Examples ...}

#The Given Prompt#:

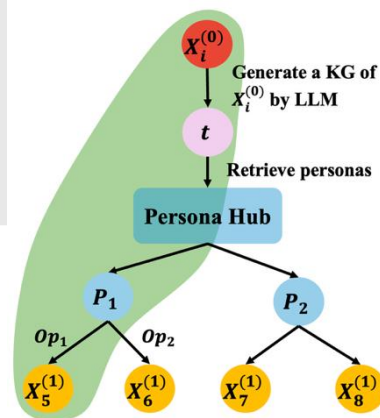
{Here is a given prompt.}

#The Given Persona#:

{Here is a related persona (retrieved).}

#Created Prompt#:

Figure 7. An example about K -hop Synthesis with personas.



#The Given Prompt#: Generate a list of ten items a person might need for a camping trip.

#Topic#: outdoor activities

#Related knowledge#: camping

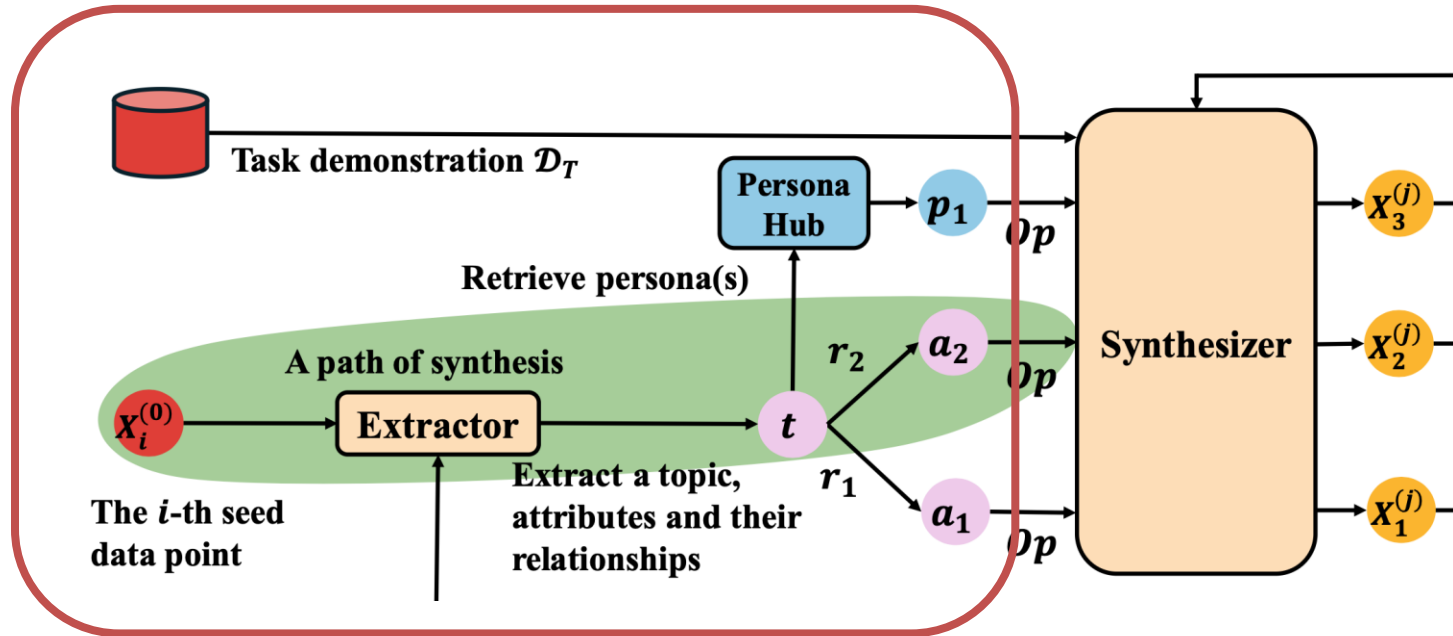
Retrieve a topic-related persona

Persona: an adventurous senior citizen who can recall some related experiences of living in high elevation.

1-hop synthesis using a persona with an operation “adding constraint”.

What are the top essential items recommended by a survival expert for a successful camping trip in harsh weather conditions?

K -hop Synthesis with Attributes and Personas Guides



+ Residual Connection Mechanism

- K -hop synthesis with guides still has an issue that relevance is reduced as the value of K become large.

An example of 10-hop synthesis

Input: Create an example of **an economic** and **a political issue**.

1-hop neighbor: Design a scenario that incorporates both **an economic** and **a political quandary**, and introduce a critical factor that significantly influences the outcome.

.....

10-hop neighbor: Construct a multifaceted narrative that intricately weaves together various **economic**, **political**, and **sociocultural elements**, entwined with an unexpected turn of events that dramatically shifts the trajectory, demanding a methodical strategy to maneuver through the complex array of repercussions, emphasizing an in-depth exploration of **ethical dilemmas**, alongside an additional dimension probing into the subconscious drivers behind the choices made by each character, all while taking into account the impact of **technological advancements** and how they shape the development of the storyline.

Figure 8. An example of 10-hop synthesis. When K is large in K -hop synthesis (i.e., $K = 10$), more irrelevant information can be introduced in the synthetic data.

Residual Connection Mechanism

- We add a residual connection between an input data and its $(K - 1)$ -hop neighbors, when K is larger threshold of depth L ,

$$X_i^{(K)} = \begin{cases} LLM(X_i^{(K-1)}, \langle t, r, k \rangle_i^{(K-1)}, Op) & K \leq L \\ LLM(X_i^{(K-1)}, \langle t, r, k \rangle_i^{(K-1)}, Op, X_i^{(0)}) & K > L \end{cases} \quad (5)$$

- We observe the necessity of residual connections due to the reduction of relevance from K -hop synthesis when K is large.

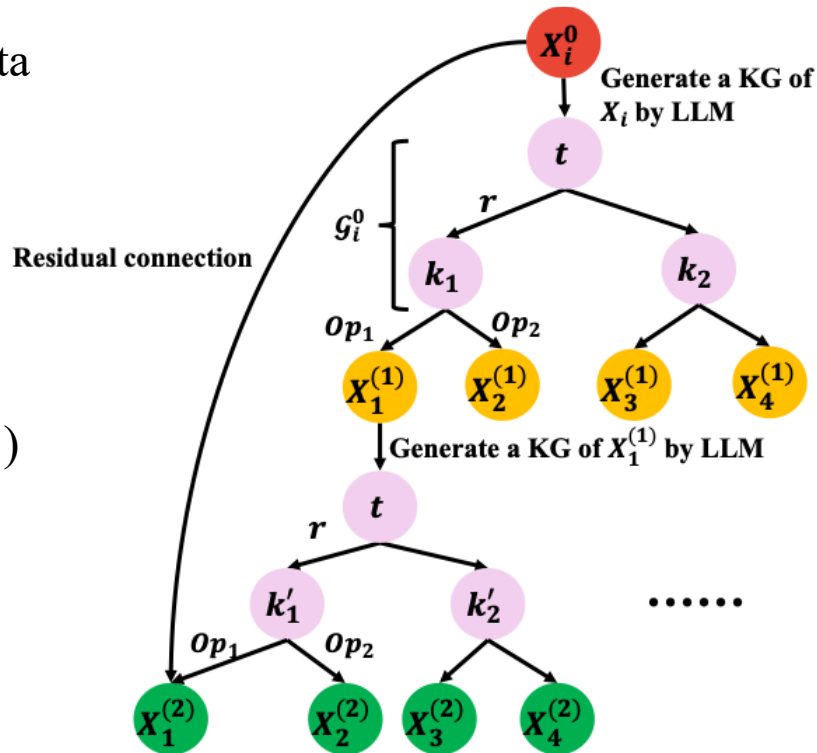


Figure 9. K-hop synthesis with KGs and predefined operations using residual connection.

Residual Connection Mechanism

An example of 10-hop synthesis with the residual connection

Input: Create an example of **an economic** and **a political issue**.

1-hop neighbor: Develop a multifaceted scenario encompassing interconnected **economic** and **political challenges**, each influencing the other in a complex and nuanced manner.

.....

10-hop neighbor: Craft an engaging narrative interlacing complex **economic** and **political dilemmas**, highlighting their symbiotic nature and profound impact on each other, necessitating a nuanced comprehension of their intricate interdependencies for adept navigation.

Figure 10: An example of 10-hop synthesis with the residual connection. Adding residual connections when $K > 2$.

K-hop Synthesis with Attribute KGs and Personas using Residual Connection

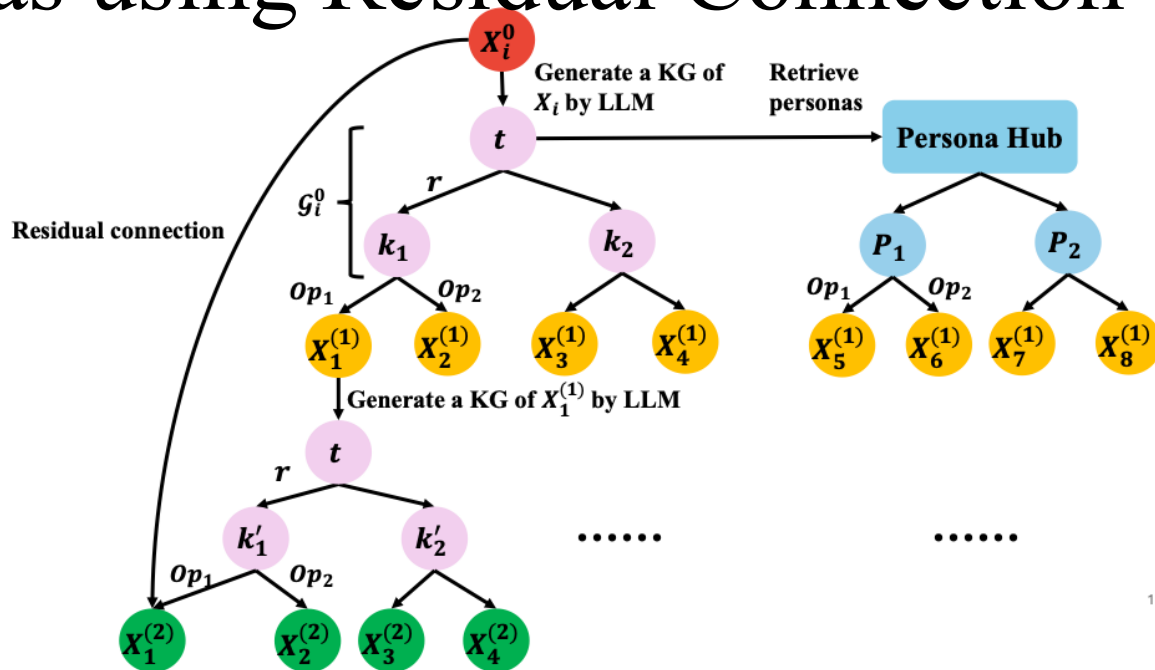


Figure 11. An example of K -hop synthesis when $K = 2$. $X_i^{(0)}$ is one of the seed data and $X^{(1)} = \{X_1^{(1)}, X_2^{(1)}, \dots, X_8^{(1)}\}$ represents synthetic data from 1-hop synthesis while $X^{(2)} = \{X_1^{(2)}, X_2^{(2)}, \dots, X_{m_2}^{(2)}\}$ represents synthetic data from 2-hop synthesis. r is the relation between t and k and Op is the abbreviation of operations.

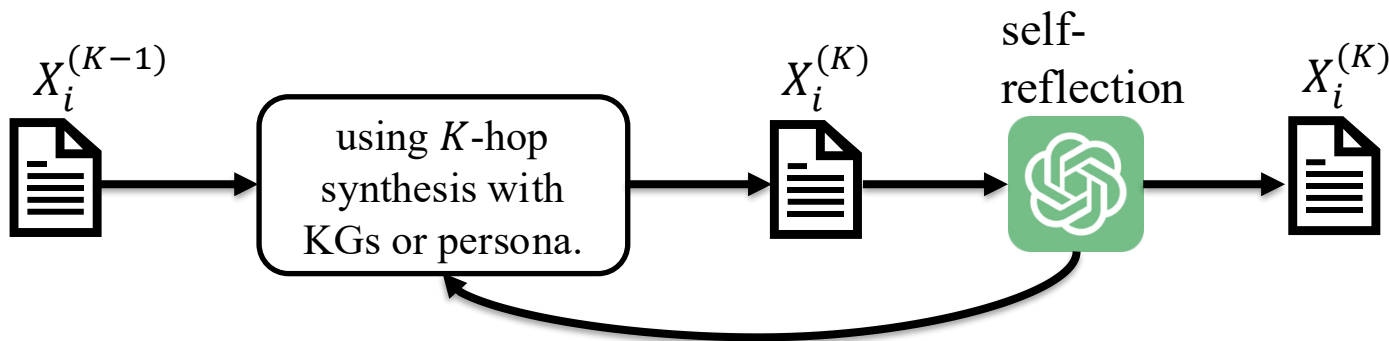
Method: Components in AIDE

AIDE combines multi-hop synthesis, attribute/persona guidance, residual connections, and self-reflection.

- **Multi-Hop Synthesis:** Recursively expand each seed — each generated point becomes a new seed for the next hop
- **Attribute Guidance:** Extract knowledge triplets ⟨topic, relation, attribute⟩ to keep synthesis on-task
- **Persona Guidance:** Retrieve diverse personas from Persona Hub to diversify synthesis paths
- **Residual Connections:** Anchor deep hops back to the original seed to prevent topic drift at $K \geq 5$
- **Self-Reflection:** LLM grader scores relevance and diversity; low-scoring outputs are re-synthesized

Self-Reflection for Improvement

- For ensuring the relevance, diversity and correctness of synthetic data, we also adapt the self-reflection technique^[8, 9] in AIDE synthesis method.
- We enable a LLM to label the synthetic data with the same self-reflection mechanism, which ensures the correctness of the labeling.



Limited number of improvement with feedback from self-reflection (i.e., improvement at most 5 times).

Figure 12. a workflow of self-reflection in our framework

Summary of AIDE

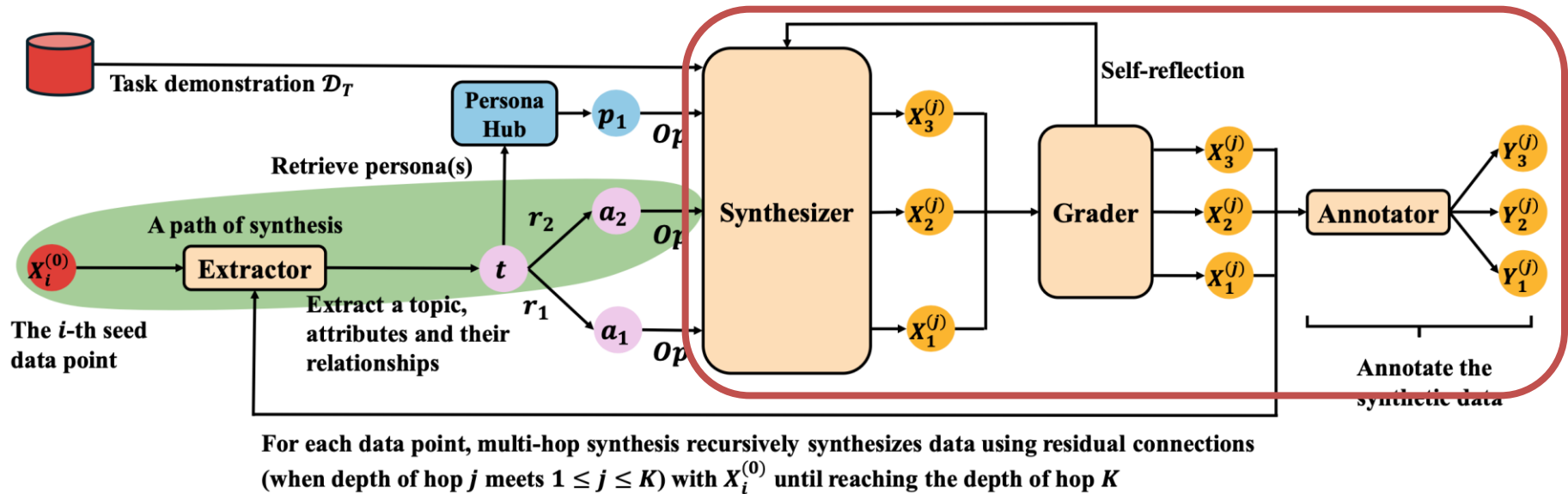


Figure 1: Overview of the workflow of AIDE. $x_i^{(0)}$ denotes the i -th task-related seed data point. AIDE includes four steps. (1) a LLM extractor extracts a topic t , knowledge attributes a_1 and a_2 with relationships r_1 and r_2 of a data point. (2) During the multi-hop synthesis at the depth of hop j , a LLM acts as a synthesizer with task demonstrations $\mathcal{D}_T$ to generate data $x_1^{(j)}$, $x_2^{(j)}$ and $x_3^{(j)}$ along paths of synthesis with a predefined operation Op (i.e., adding constraints). (3) To enhance the diversity of synthesis, we expand attributes by retrieving a persona p_1 from a persona hub with t . Finally, a LLM as an annotator generates the label of synthetic data. We describe the technical details of AIDE in Section 3.

Experiments to answer:

- (RQ1) Can AIDE enable the fine-tuning of pretrained models that outperform those fine-tuned on human-curated data and data generated by SOTA synthesis methods?
- (RQ2) How does AIDE affect pretrained models' performance under different settings?
- (RQ3) Does the data from AIDE maintain relevance and diversity?

Five Baselines:

- **Gold (Human-Curated):** Real annotated training examples — the practical upper-bound reference
- **Evol-Instruct:** Complexity-driven open-domain synthesis using 5 operations; trained with 250,000 synthetic points
- **DataTune:** Retrieves and transforms existing datasets for a target task description
- **Prompt2Model:** Generates deployable models from natural language task instructions
- **Pretrained (no fine-tuning):** Zero-shot baseline with no task adaptation whatsoever

Experiments on 15 tasks across 3 model families with 10 seed examples per task.

- **Models:** Mistral-7B, Llama-3.1-8B, Llama-3.2-3B — fine-tuned with LoRA ($r=8$, $\alpha=16$)
- **Benchmarks:** BIG-Bench (5 tasks), MMLU (5 tasks), **ARC-Challenge**, **GSM8K**, **TruthfulQA**, **MedQA**, **FinBen-CFA**
- **Metric:** Zero-shot accuracy; 8-shot maj@8 for GSM8K
- **AIDE config:** K=2 hops, 10 seeds $\rightarrow$ $\sim$ 3,000 synthetic examples; Claude Sonnet 3.5 as generator/synthesizer
- **Hardware:** 8 $\times$ NVIDIA A100 GPUs, AdamW optimizer, 10 epochs with batch size 10

Generated AIDE datasets:

Benchmarks	Task Name	Depth of K	Amount of seed Data	Quantity of Synthetic Data
BIG-Bench	Code	2	10	3.0K
	C&E	2	10	3.2K
	Impl.	2	10	3.1K
	Math	2	10	3.1K
	Time	2	10	3.2K
MMLU	Bio.	2	10	3.4K
	CS	2	10	3.2K
	Phi.	2	10	3.4K
	EE	2	10	3.0K
	Market.	2	10	3.3K
ARC-Challenge	-	2	10	3.3K
GSM8K	-	2	10	3.2K
TruthfulQA	-	2	10	3.1K
FinBen	CFA	2	10	893
MedQA	-	2	10	2.2K

Models fine-tuned on 10-seed AIDE data consistently outperform models trained on real human-labeled examples.

Model	AIDE Avg.	Gold Avg.	Δ Improvement
Mistral-7B	64.3%	60.1%	+7.0%
Llama-3.1-8B	63.9%	63.7%	+0.7%
Llama-3.2-3B	55.9%	55.1%	+1.5%

AIDE Outperforms All Synthesis Baselines by a Large Margin

Method	BIG-Bench Avg.	Data Used
AIDE (Ours)	74.2%	10 seeds
Evol-Instruct	54.2%	250,000 seeds
DataTune	35.2%	Multiple seed datasets
Prompt2Model	36.1%	Multiple seed datasets
Pretrained (no FT)	33.7%	—

AIDE exceeds Evol-Instruct by >20% , though Evol-Instruct using 25,000× more data points

(RQ1)

More Details:

Pretrained Model	Fine-tuning with Data Source	BIG-Bench					Avg. ($\uparrow$)
		Code	C&E	Impl.	Math	Time	
Mistral-7B	AIDE (Ours)	91.7%	99.2%	67.9%	21.0%	90.3%	74.2%
	Prompt2Model	84.5%	41.2%	48.0%	4.7%	2.0%	36.1%
	DataTune	73.4%	33.8%	44.0%	8.1%	16.9%	35.2%
	Evol-Instruct	73.3%	73.2%	65.1%	14.1%	45.2%	54.2%
	Pretrained Model	46.7%	47.7%	61.1%	11.6%	1.4%	33.7%

Table 3: AIDE vs. SOTA Data Synthesis Methods. We compare the performance of various zero-shot learning approaches in Mistral-7B fine-tuned with AIDE and SOTA synthesis methods across five BIG-Bench tasks. The table follows a setup similar to Table 2. Notably, Evol-Instruct fine-tunes Mistral-7B with 250K synthetic data points.

(RQ1)

Models fine-tuned on 10-seed AIDE data consistently outperform models trained on real human-labeled examples.

	Fine-tuning with Data Source	MMLU					FinBen	ARC- Challenge	GSM8K	TruthfulQA	MedQA	Avg. (↑)	Avg. Δ (↑)
		Bio.	CS	Phi.	EE	Market.	CFA						
# Seed Data Points in AIDE		10	10	10	10	10	10	10	10	10	10	10	
Pretrained Mistral-7B	AIDE (Ours)	75.5%	57.0%	72.2%	60.7%	89.3%	41.0%	74.7%	59.1%	69.2%	44.0%	64.3%	7.0%
	Gold training data	73.2%	56.0%	71.1%	60.0%	85.9%	35.0%	79.4%	53.4%	49.9%	37.0%	60.1%	NA
Pretrained Llama-3.1-8B	AIDE (Ours)	74.2%	47.0%	63.0%	49.7%	82.1%	62.0%	69.8%	65.8%	69.2%	56.0%	63.9%	0.7%
	Gold training data	74.7%	48.1%	60.5%	50.1%	82.3%	61.0%	69.6%	68.2%	66.1%	54.0%	63.7%	NA
Pretrained Llama-3.2-3B	AIDE (Ours)	58.7%	43.4%	56.6%	54.5%	71.4%	54.0%	56.8%	45.1%	67.6%	51.0%	55.9%	1.5%
	Gold training data	60.2%	45.0%	55.6%	48.3%	70.7%	54.0%	56.5%	45.5%	64.9%	50.0%	55.1%	NA

Table 2: AIDE-generated data vs. human-curated training data for fine-tuning. We evaluate the performance of various

(RQ2)

Ablation Study:

Removing any single component causes a meaningful performance drop — all three work together.

Ablation on BIG-Bench Time Task (Mistral-7B)

All three together achieve 90.3% — the full combination is essential.

Attributes	Personas	Residual	Accuracy
✓	✗	✗	60.1%
✗	✓	✗	49.3%
✓	✓	✗	72.2%
✓	✗	✓	75.0%
✓	✓	✓	90.3%

(RQ2)

Ablation Study:

Attribute guidance, task demonstrations, and synthesizer quality are the most critical levers.

- **Attributes > Personas alone:** Attribute paths (60.1%) outperform persona-only (49.3%) — topic control matters more than diversity alone
- **Task demonstrations matter:** Including task-related examples in synthesis boosts accuracy by >10% vs. AIDE without demonstrations
- **More seeds → better diversity:** Performance improves consistently as seed count grows from 5 → 10 → 20
- **Synthesizer quality is flexible:** Both Claude Sonnet 3.5 and GPT-3.5-Turbo yield >40% gains over pretrained — the AIDE framework generalizes across LLMs
- **Math reasoning hits a ceiling:** Zero-shot math stays ~20% — Chain-of-Thought integration is needed for multi-step reasoning tasks

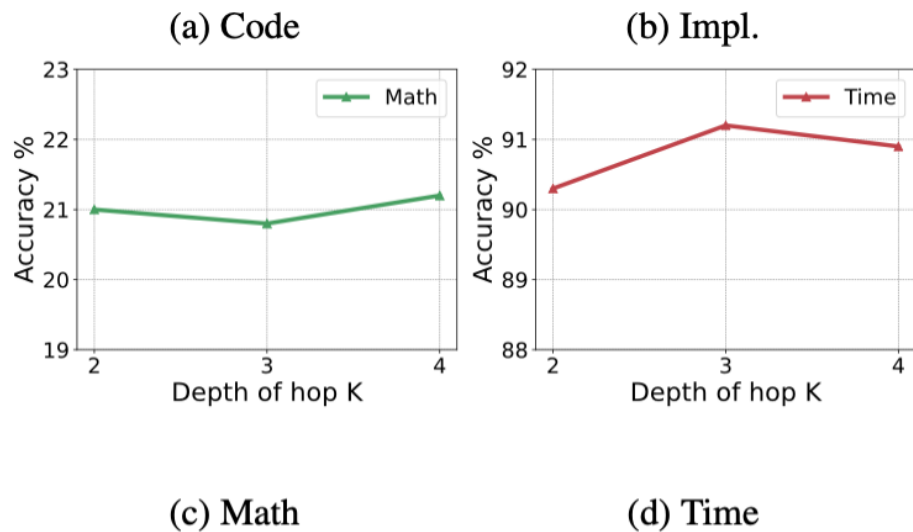
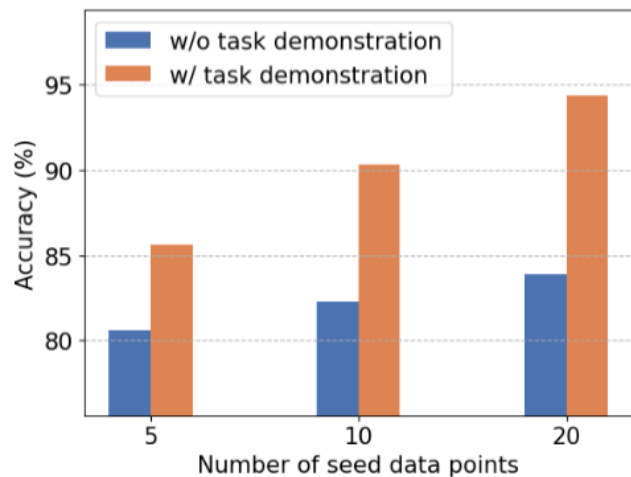


Figure 3: Effect of data quantity with different number of K values in multi-hop synthesis based on the BIG-Bench.

Figure 2: The effect of varying the number of seed data w/ and w/o task demonstration on the Time task from BIG-Bench.

AIDE-generated data matches gold data on diversity while maintaining strong task alignment.

- **Relevance (cosine sim.):** Average similarity between synthetic and seed data exceeds 0.5 across all tasks
- **Relevance (LLM-judged):** Claude Sonnet 3.5 assigns relevance scores of 5–9/10 on average across Code, C&E, and Impl. tasks
- **Diversity (Self-BLEU):** AIDE Self-BLEU scores are comparable to or better than gold data on 11 of 13 tasks
- **Complexity:** AIDE achieves complexity scores comparable to Evol-Instruct using only 3 operations (vs. 5)
- **Coverage (t-SNE):** AIDE synthetic data covers the full distribution of real test data in embedding space

(RQ3)

Benchmarks	Task Name	Diversity of Synthetic Data (AIDE)	Diversity of Gold Data
BIG-Bench	Code	0.59	0.50
	C&E	0.21	0.15
	Impl.	0.43	0.40
	Math	0.49	0.50
	Time	0.70	0.91
MMLU	Bio.	0.41	0.29
	CS	0.66	0.24
	Phi.	0.49	0.30
	EE	0.60	0.18
	Market.	0.44	0.25
ARC-Challenge	-	0.43	0.18
GSM8K	-	0.43	0.21
TruthfulQA	-	0.67	0.20

Table 6: Quantitative comparison of diversity between synthetic data from AIDE for different tasks and gold data from different tasks. We **highlight lower Self-BLEU scores**, which implies higher diversity.

Summary: AIDE enables competitive task-specific LLM fine-tuning using only a handful of examples.

- **Low-resource fine-tuning is viable:** 10 seeds are sufficient to fine-tune competitive models across medicine, finance, reasoning, and code
- **Synthetic can surpass human-labeled:** AIDE outperforms gold data by up to 7% — structured LLM synthesis beats manual annotation
- **Modular and extensible:** Each component (attributes, personas, residual, self-reflection) is independently improvable
- **Open source:** Code available at github.com/Code4Graph/AIDE

Reference

- [1] Stanford alpaca: An instruction-following llama model. 2023, (https://github.com/tatsu-lab/stanford_alpaca).
- [2] WizardLM: Empowering large pre-trained language models to follow complex instructions. ICLR, 2024
- [3] The flan collection: designing data and methods for effective instruction tuning. ICML, 2023
- [4] ZeroPrompt: Scaling prompt-based pretraining to 1,000 tasks improves zero-shot generalization. Findings of EMNLP, 2022
- [5] Scaling instruction-finetuned language models, JMLR, 2024.
- [6] Scaling Data Diversity for Fine-Tuning Language Models in Human Alignment. LREC-COLING, 2024
- [7] Scaling Synthetic Data Creation with 1,000,000,000 Personas. 2024, (<https://arxiv.org/abs/2406.20094>).
- [8] Self-refine: Iterative refinement with self-feedback. NeurIPS, 2023.
- [9] Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies. ACL, 2024.
- [10] Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. TMLR, 2023.
- [11] The llama 3 herd of models. 2024, (<https://arxiv.org/abs/2407.21783>).
- [12] Mistral 7b. 2023, (<https://arxiv.org/abs/2310.06825>).
- [13] LoRA: Low-rank adaptation of large language models. ICLR, 2022.
- [14] Decoupled weight decay regularization. ICLR, 2019.
- [15] Prompt2Model: Generating Deployable Models from Natural Language Instructions. 2023, (<https://arxiv.org/abs/2308.12261>)
- [16] Better Synthetic Data by Retrieving and Transforming Existing Datasets. 2024, (<https://arxiv.org/abs/2404.14361>)

DFLOW: Diverse Dialogue Flow Simulation with Large Language Models

Wanyu Du · Song Feng · James Gung · Lijia Sun · Yi Zhang · Saab Mansour · Yanjun Qi

REALM 2025 · Association for Computational Linguistics · July 31, 2025

DFLOW: Diverse Dialogue Flow Simulation with Large Language Models

Wanyu Du* Song Feng James Gung

Lijia Sun Yi Zhang Saab Mansour Yanjun Qi

Amazon Web Services

{dwanyu, sofeng, gungj}@amazon.com

{sunlijia, yizhngn, saabm, yanjunqi}@amazon.com

SDG Prompting with Rich Prompts:

Common Structure of LLM Prompts:

- [System/Role]
- [Task Instruction]
- [Context/Background]
- [Input Data]
- [Few Shot Examples]
- [Output Format]
- [Constraints]

**AIDE: Atttribute-Guided MultI-Hop Data Expansion
for Data Scarcity in Task-Specific Fine-tuning**

Jiayu Li^{1,3,*}, Xuan Zhu², Fang Liu², Yanjun Qi^{2,3}

¹Syracuse University, ²AWS Bedrock Science

³Correspondence: jli221@data.syr.edu, yanjunqi@amazon.com

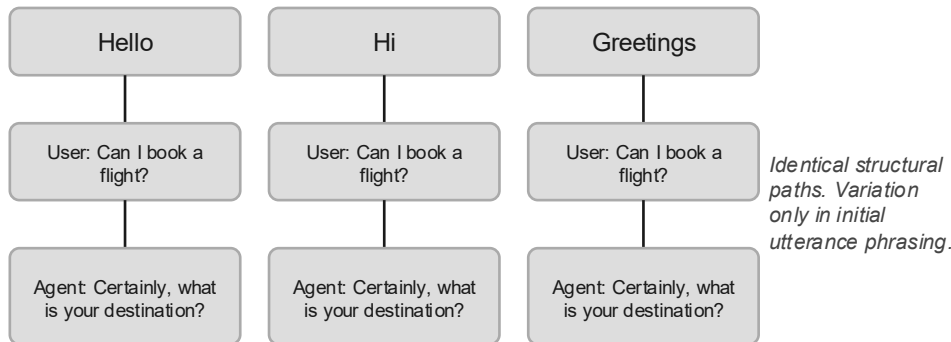
Background:

- A task-oriented LLM agent typically needs to manage user requests according to **a designated "plan guide"** which includes predefined task logics and policy constraints.
- **Problem:** Existing dialogue datasets lack diversity in **task plan logic** (dialogue flows), focusing mostly on surface-level text variation, which limits LLM agents' ability to handle complex or unexpected scenarios.
- Manually curating such dialogue flows is challenging due to the intricate task logic and alignment with policy constraints across various domains.
 - **Gap: no scalable method to generate logically coherent dialogue flows automatically**

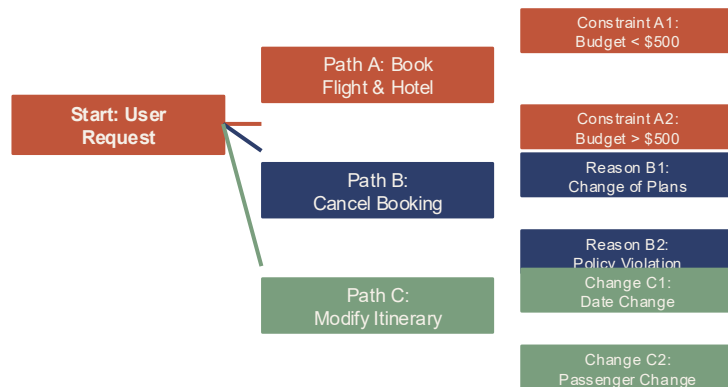
Current synthetic datasets diversify words, not trajectories

High linguistic diversity, Low logic diversity.

Utterance-Level Diversity



Dialogue Flow-Level Diversity



Need an automated mechanism to generate task-oriented dialogues grounded in structural logic.

DFLOW: one task instruction produces many diverse dialogue flows via a three-step pipeline

- **Step 1 — Task Plan Generation**
 - LLM Planner reads a task instruction and outputs a decision-tree task plan
- **Step 2 — Dialogue Flow Sampling**
 - Depth-First Search extracts every root-to-leaf path as one unique dialogue flow
- **Step 3 — Dialogue Generation**
 - LLM Synthesizer uses each flow as a scaffold to generate a faithful multi-turn dialogue
- **One instruction → many flows → many coherent training dialogues**



Step 1: Task Plan Generation



Task Pool

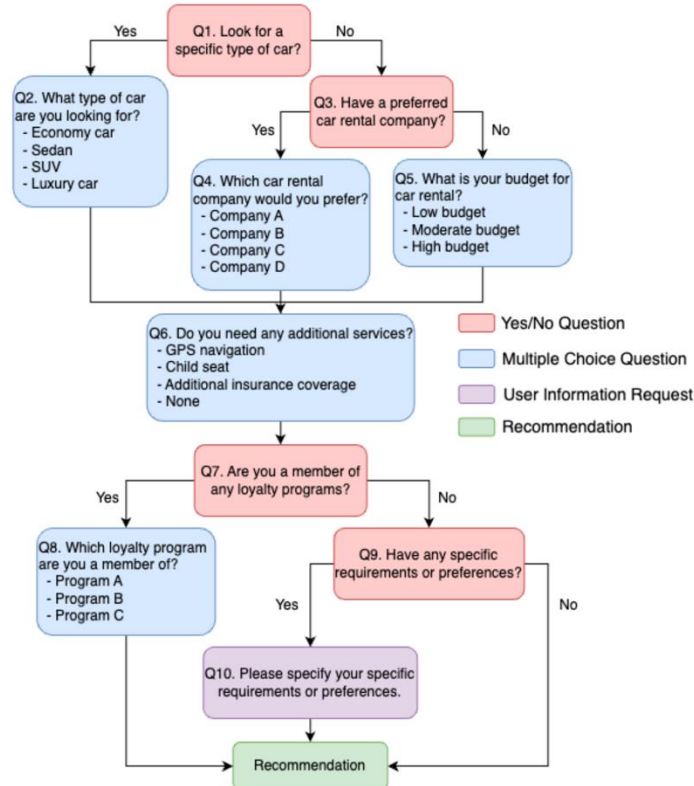


LLM
Planner

Domain: Car Rental

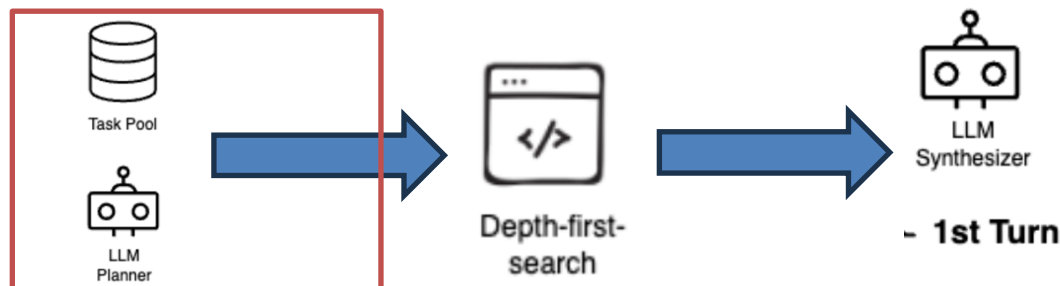
Task Instruction: Please provide an example of decision tree for "Explore the various car rental services offered".

Task Plan:



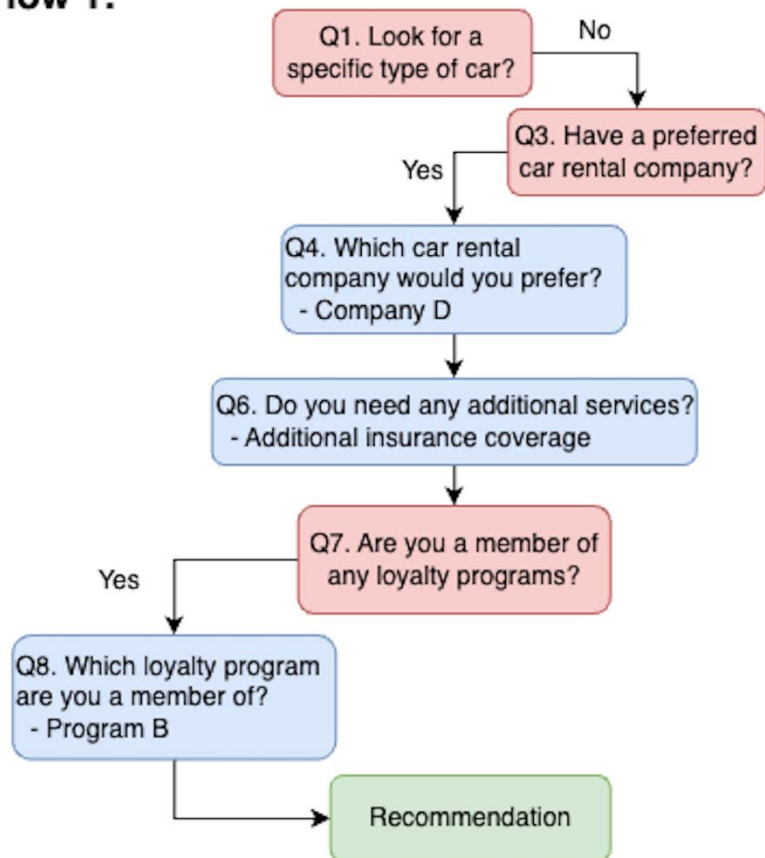
Step 1: The LLM Planner generates a decision tree that encodes every possible task path

- Input: a task instruction (e.g. "Explore car rental services")
- Output: a decision tree — each node is one of four system action types:
 - Yes/No Questions — binary branching to different task paths
 - Multiple Choice Questions — multi-way branching to next steps
 - User Information Requests — open-ended slot-filling collection
 - Recommendations — leaf nodes marking task completion
- **Every root-to-leaf path in the tree = one unique task trajectory**
- Four different LLMs generate plans per instruction for added variety



Step 2: Dialogue Flow Sampling

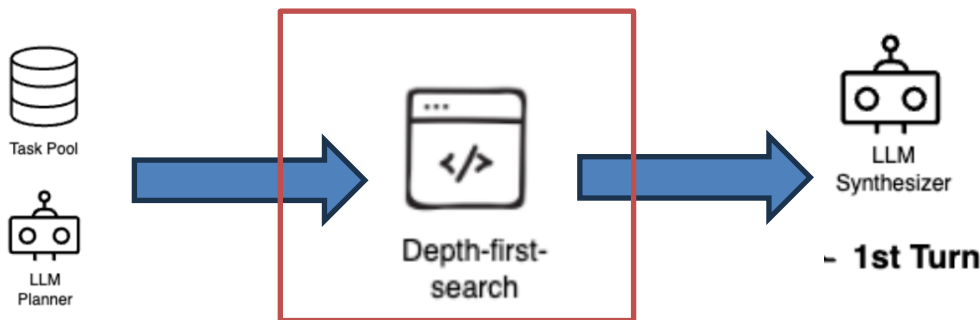
Flow 1:



Depth-first-
search

Step 2: Depth-First Search extracts all valid paths — including real-world error cases

- Each root-to-leaf path = one dialogue flow (ordered sequence of system actions + values)
- Normal flows cover all standard happy-path user journeys
- **Two special error-handling flows are added per plan:**
 - Out-of-scope Request Flow — user gives invalid input; agent detects and corrects
 - Early-stop Flow — user abandons the task before completion
- Error-handling flows capture real-world edge cases that happy-path datasets miss



Step 3: Flow Controlled Dialogue Generation

Dialogue 1:

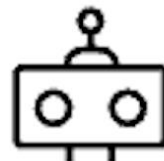
Hi there! I'm interested in renting a car for my upcoming trip. Do you have any available? (Q1)

Hello! Of course, I'd be happy to assist you with that. Before we proceed, **do you have a specific type of car in mind?** (Q1)

I'm actually open to exploring different options. So, **no** specific type in mind. (Q1)

Got it. That gives us more flexibility. **Do you have a preferred car rental company** you'd like to go with? (Q3)

.....



LLM
Synthesizer

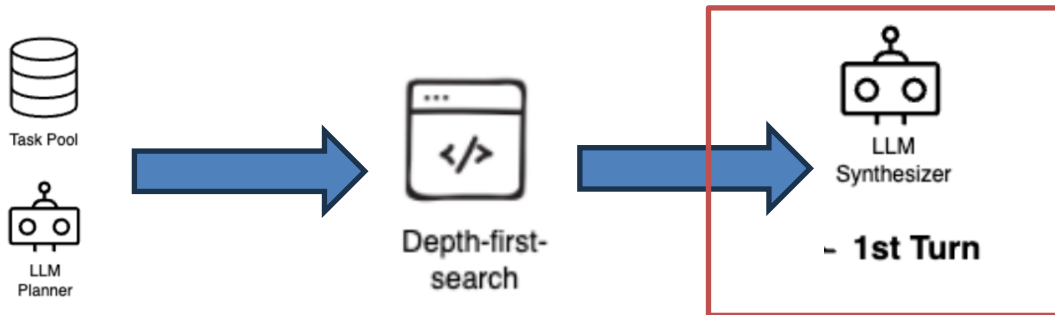
1st Turn

2nd Turn

.....

Step 3: Each flow controls an LLM Synthesizer to generate a faithful multi-turn dialogue

- The dialogue flow acts as a structural scaffold — each turn maps to one flow step
- An automatic quality filter removes utterances not linked to any flow step
- Repetitive or contradictory utterances are also automatically removed
- **Four LLMs synthesize dialogues for model diversity:**
 - GPT-3.5-turbo · Mistral-8x7B · Falcon-40B · MPT-30B-chat
- All 3,886 flows → 3,886 dialogues, avg. 8.83 turns each



Summary

Dataset Scale & Variety: Using LLMs as planners and generators, DFLOW produces thousands of multi-domain, multi-style dialogues with error-handling and alternate paths, far surpassing the coverage of manually annotated datasets.

Key Idea: DFLOW introduces a **flow-centric synthetic dialogue generation framework**, automatically creating diverse and logic consistent dialogue flows before generating human-agent conversations.

Contributions: (1) A scalable data simulation framework, (2) a diverse dialogue dataset, (3) a benchmark for next-action prediction, and (4) empirical evidence that fine-tuning smaller models on DFLOW improves performance beyond GPT-4 on this benchmark.

DFLOW data statistics:

	Train	Test	Total
# Domains	10	5	15
# Task Instructions	100	30	130
# Plans	293	101	394
# Flows	3,229	657	3,886
# Dialogues	3,229	657	3,886
# Utterances	29,342	5,634	34,976
Avg. Plan Steps	9.66	8.16	9.39
Avg. Flow Steps	7.98	6.78	7.76
Avg. Dialogue Turns	9.03	7.89	8.83

Table 2: Statistics of our DFLOW dataset, where domains in the test set have no overlap with the train set.

Intrinsic Evaluation: DFLOW data statistics from Four LLMs:

	GPT-3.5-turbo	Mixtral-8x7B-instruct	Falcon-40B-instruct	MPT-30B-chat
# Domains	15	15	15	15
# Task Instructions	130	130	130	130
# Plans	110	98	83	103
# Flows	1,192	1,222	765	781
- # Normal Flows	916	1,118	559	578
- # Error Handling Flows	276	104	206	203
# Dialogues	1,192	1,222	765	781
# Utterances	11,015	8,784	7,448	7,729
Avg. Plan Steps	9.33	8.68	10.95	9.07
Avg. Flow Steps	7.78	6.29	9.73	8.10
Avg. Dialogue Turns	9.24	7.18	9.73	9.89

Table 1: Statistics of DFLOW generated from four LLMs.

Intrinsic Evaluation: Quality Evaluation of DFLOW data

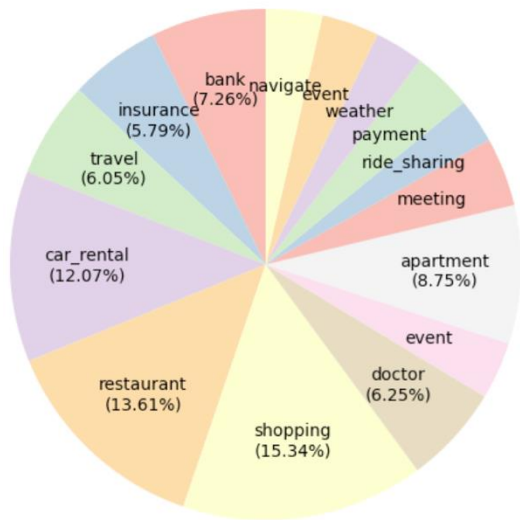
	DFLOW (flow by auto)	ABCD (flow by human)
Plan Quality	0.9835	0.9943
Plan Coherence	0.9182	1.0000
Flow Coherence	0.9220	1.0000
Dialogue Faithfulness	0.8718	0.8011

Table 3: Data quality evaluation on DFLOW and ABCD. GPT-4 evaluation results on all data pairs <plan, flow, dialogue> from DFLOW (full) and ABCD (test).

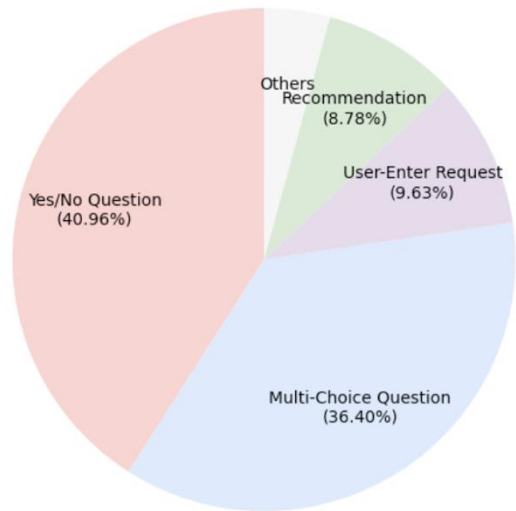
DFLOW vs. expensive human annotation (ABCD)

- achieves parity on structural coherence, and actually
- surpasses human baselines in ensuring the text dialogue faithfully matches the intended logic flow.

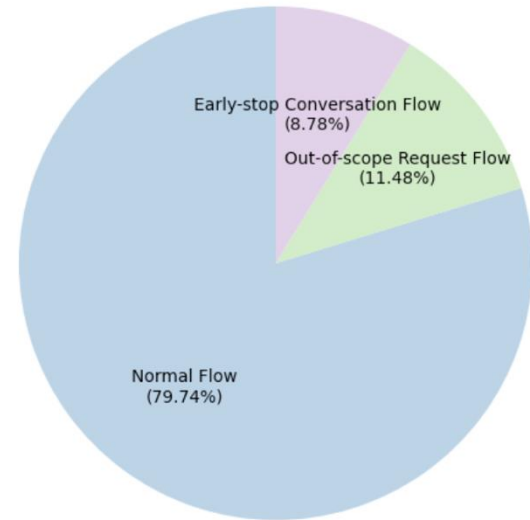
Intrinsic Evaluation: Data Diversity



(a) Task domain distribution



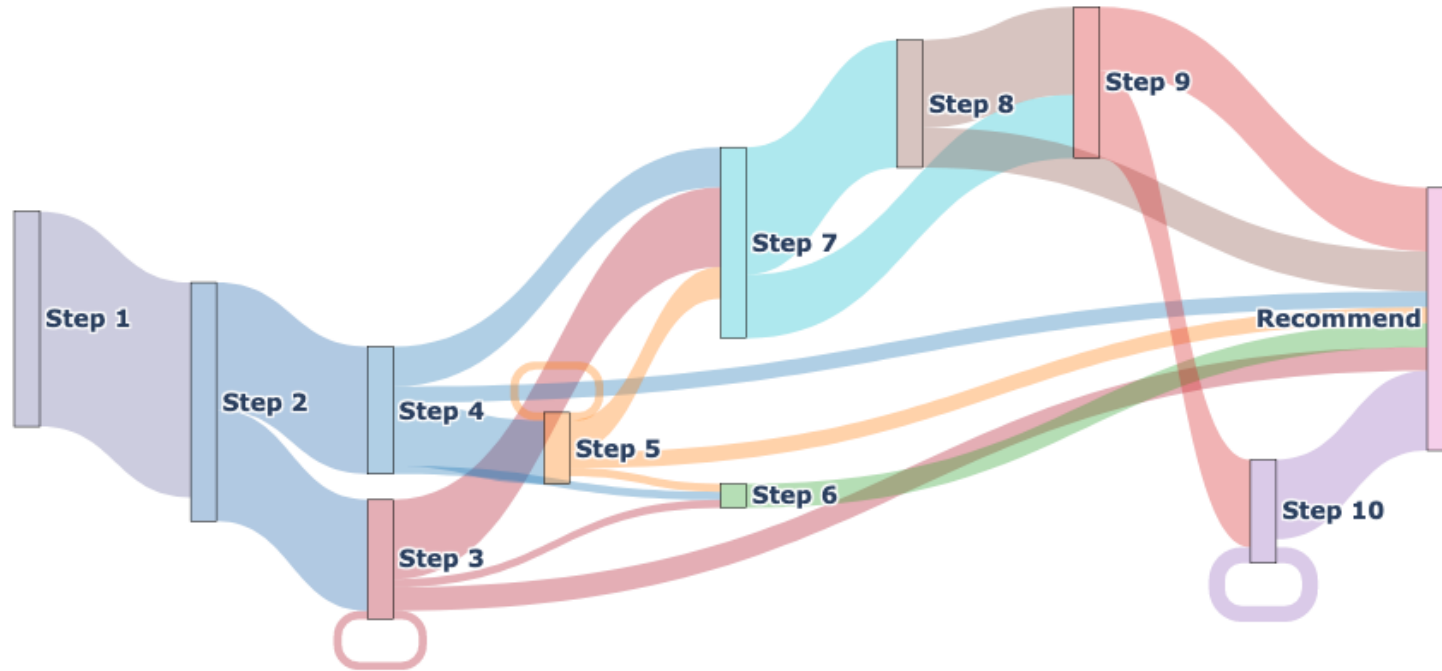
(b) Plan action distribution



(c) Flow type distribution

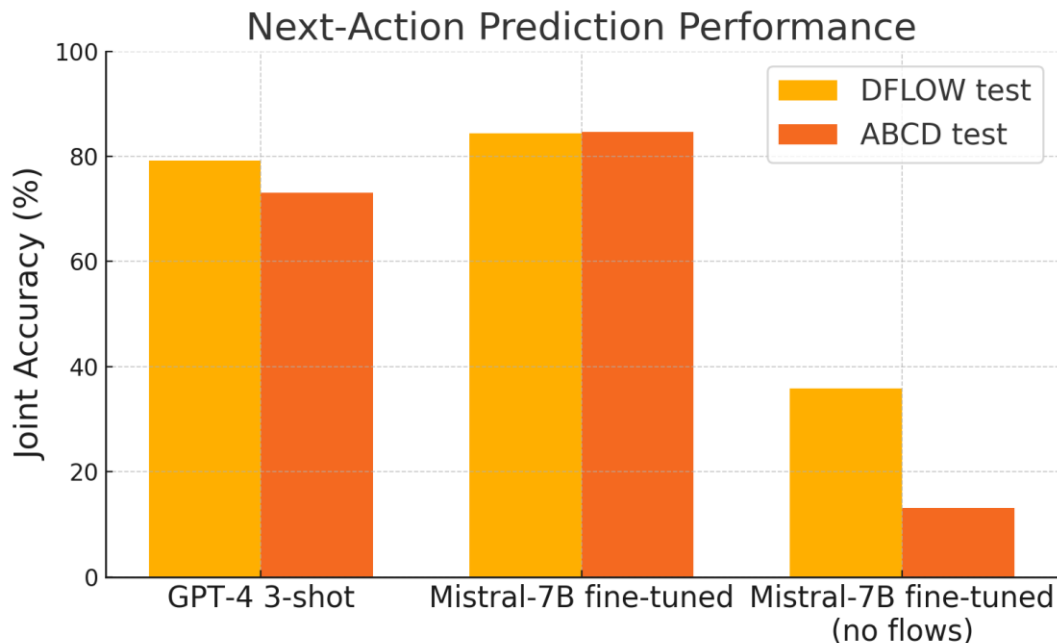
Figure 3: Diverse domain, plan action type and flow type distribution of our DFLOW dataset.

Intrinsic Evaluation: Flow Diversity



- 35 dialogue flows from "Agenda creation" plan generated by Mistral-8x7B-instruct

Extrinsic Evaluation:



- **Effectiveness of DFLOW fine-tuning:** A *Mistral-7B* model fine-tuned on DFLOW achieves **84 % joint-action accuracy** on the DFLOW test set and **85 % on the ABCD transfer set**, surpassing a *GPT-4* 3-shot baseline by approximately 5 and 11 percentage points, respectively.
- **Ablation underscores the importance of flows:** Removing dialogue-flow supervision from the fine-tuning reduces joint-action prediction accuracy to **36 % on DFLOW** and **13 % on ABCD**. This substantial drop confirms that the diversity and policy compliance encoded in DFLOW's generated flows are critical to the performance.

Extrinsic Evaluation: Next Action Prediction

Setup	Model	DFLOW TEST			ABCD TEST (HUMAN)		
		Action	Value	Accuracy	B-Slot	Value	Accuracy
0-shot	GPT-4	73.72%	77.99%	73.29%	70.46%	87.82%	70.26%
	Mistral-large-2402	66.67%	60.29%	55.34%	58.68%	78.04%	57.29%
	Mistral-7B-instruct-v0.2	39.10%	29.70%	23.50%	43.91%	65.27%	41.92%
	OpenLlama-7B-instruct	14.10%	14.32%	9.19%	27.74%	51.70%	22.55%
3-shot	GPT-4	79.70%	82.26%	79.27%	73.25%	86.03%	73.05%
	Mistral-large-2402	72.86%	76.50%	72.44%	62.67%	79.04%	61.48%
	Mistral-7B-instruct-v0.2	39.53%	38.46%	30.56%	43.11%	65.67%	42.32%
	OpenLlama-7B-instruct	22.01%	24.36%	19.44%	32.33%	56.69%	31.14%
Fine-tune (with DFLOW)	Mistral-7B-instruct-v0.2	84.40%	85.47%	84.40%	84.83%	92.42%	84.63%
	OpenLlama-7B-instruct	73.29%	74.79%	73.08%	87.62%	93.41%	86.63%
Fine-tune (w/o flow) (Ablation)	Mistral-7B-instruct-v0.2	39.96%	41.45%	35.90%	17.96%	50.10%	13.17%
	OpenLlama-7B-instruct	36.11%	37.61%	25.43%	26.35%	53.89%	24.95%

Table 5: Next action prediction performance on the test set of DFLOW and ABCD, where both fine-tune setups use the training set of DFLOW to fine-tune models. For DFLOW and ABCD, the domains and tasks in test set have no overlap with the training set.

Experimental Setup

- **Dataset: 15 domains, 130 task instructions, 3,886 flows, 34,976 utterances total**
- Fine-tuning: LoRA rank=16, $\alpha=32$, lr= 2×10^{-4} , 6 epochs, 4× NVIDIA A10G GPUs
- For ABCD evaluation: 290 ABCD training samples added to teach output format only
- **Metrics: Action Accuracy · Value Accuracy · Joint Accuracy**
- Task: Next Action Prediction — predict the correct system action + slot values given dialogue context + flow
- Test sets: DFLOW (5 unseen domains, 657 dialogues) and ABCD (500 human-annotated samples)

Ablation: Each LLM shows a distinct bias — multi-LLM synthesis is necessary for coverage

LLM	Yes/No Q	Multi-Choice Q	Info Request	Key Bias
GPT-3.5-turbo	43.6%	34.9%	9.4%	Prefers Yes/No Qs
Mistral-8x7B	42.5%	33.3%	6.6%	More 'Other' types
Falcon-40B	40.8%	36.4%	13.9%	More Info Requests
MPT-30B-chat	34.5%	43.5%	9.9%	Prefers Multi-Choice Qs

- **No single LLM covers all action-type distributions uniformly.**
- Combining 4 LLMs smooths individual biases and generates richer, more balanced training data.

Conclusion:

Method: DFLOW proposes a dialogue simulation framework that leverages task plans to synthesize diverse dialogue flows.

Dataset: DFlow dataset spans 15 domains with 4K dialogues, forming the a high-quality benchmark for the next-action prediction task.

Insight: Fine-tuning on high-quality and diverse datasets helps small language models to outperform GPT-4 in domain-specific tasks.

- **Future directions:**

- Incorporate multimodal inputs (images, graphs, tabular data) as task plan sources
- Scale to 100B+ LLMs with memory-efficient inference
- Extend to cross-domain and multilingual task-oriented dialogue settings

References:

- [Mixture of Soft Prompts for Controllable Data Generation](#) (Chen et al., Findings 2023)
- [Enhancing Chat Language Models by Scaling High-quality Instructional Conversations](#) (Ding et al., EMNLP 2023)
- [Tree of Thoughts: Deliberate Problem Solving with Large Language Models](#) (Yao et al., NeurIPS 2023)
- [Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models](#) (Wang et al., ACL 2023)
- [Scaling autonomous agents via automatic reward modeling and planning](#) (Chen et al., ICLR 2025)
- [Action-Based Conversations Dataset: A Corpus for Building More In-Depth Task-Oriented Dialogue Systems](#) (Chen et al., NAACL 2021)